

CSE 214: Creational Pattern Online Solution

January 2026 • CSE 214 • Online on Creational Pattern (Time: 20 minutes)

You are developing an online examination system. All modules, such as Student Login, Question Management, and Result Processing, must record their activities using one shared AuditLogger. Creating multiple logger objects may produce inconsistent logs and multiple output files.

Task: Implement the AuditLogger class so that only one instance can exist throughout the application.

- The logger constructor must not be directly accessible from client classes.
- The logger should be created when it is requested for the first time.
- Demonstrate access from two different modules.
- Show that both modules receive the same logger instance.
- Thread synchronization is not required.

Complete Java Implementation (`Main.java`)

Below is the complete production code inside a single compilation unit. You can copy and save this code directly into a file named `Main.java`.

```
// =====  
// 1. THE SINGLETON AUDIT LOGGER CLASS  
// =====  
class AuditLogger {  
    // Private static variable to hold the single instance  
    private static AuditLogger instance;  
  
    // Private constructor ensures no direct instantiation from client classes  
    private AuditLogger() {  
        System.out.println("--- [System Notice] AuditLogger initialized for the first  
time. ---");  
    }  
  
    // Public static method providing global access via Lazy Initialization  
    public static AuditLogger getInstance() {  
        if (instance == null) {  
            instance = new AuditLogger();  
        }  
        return instance;  
    }  
  
    // Logging API  
    public void log(String message) {  
        System.out.println("[AUDIT LOG] " + message);  
    }  
}
```

```

}

// =====
// 2. MODULES DEMONSTRATING ACCESS
// =====

class StudentLoginModule {
    private AuditLogger logger;

    public StudentLoginModule() {
        // Acquiring the shared instance
        this.logger = AuditLogger.getInstance();
    }

    public AuditLogger getLogger() {
        return this.logger;
    }

    public void login(String username) {
        logger.log("User " + username + " logged in successfully.");
    }
}

class QuestionManagementModule {
    private AuditLogger logger;

    public QuestionManagementModule() {
        // Acquiring the shared instance
        this.logger = AuditLogger.getInstance();
    }

    public AuditLogger getLogger() {
        return this.logger;
    }

    public void addQuestion(String questionId) {
        logger.log("Question ID " + questionId + " was published.");
    }
}

// =====
// 3. MAIN RUNNABLE DRIVER CLASS
// =====

public class Main {
    public static void main(String[] args) {
        System.out.println("Execution Started.");
        System.out.println();

        // Module 1 initialization triggers the lazy instance creation
        System.out.println("Instantiating StudentLoginModule...");
        StudentLoginModule loginModule = new StudentLoginModule();
        loginModule.login("shuvo_2305025");
        System.out.println();
    }
}

```

```

// Module 2 initialization handles the existing instance reference
System.out.println("Instantiating QuestionManagementModule...");
QuestionManagementModule questionModule = new QuestionManagementModule();
questionModule.addQuestion("CSE214-Q1");
System.out.println();

// Verification of instance mapping
System.out.println("--- Verifying Identity Equality ---");
AuditLogger loggerRef1 = loginModule.getLogger();
AuditLogger loggerRef2 = questionModule.getLogger();

// Reference identity check
if (loggerRef1 == loggerRef2) {
    System.out.println("SUCCESS: Both modules point to the EXACT same object instance.");
    System.out.println("Reference 1 hashCode: " +
System.identityHashCode(loggerRef1));
    System.out.println("Reference 2 hashCode: " +
System.identityHashCode(loggerRef2));
} else {
    System.out.println("FAILURE: Multiple instances detected.");
}
}
}

```

Verification of Constraints

- **Inaccessible Constructor:** The constructor is declared as private `AuditLogger()`, ensuring any external attempts to call `new AuditLogger()` generate a compilation error.
- **Lazy Initialization:** The expression `instance = new AuditLogger()` will evaluate inside `getInstance()` if and only if `instance == null`, delaying execution until the system makes its very first request.
- **Multi-module Access & Reference Proof:** `StudentLoginModule` and `QuestionManagementModule` act as separate client contexts. The main function uses the `==` reference comparison operator to confirm that both pointers target the same physical block in heap memory.